them reactive, think twice about whether that's necessary, and consider both the benefits and drawbacks.

Reactive programming is not a new concept. It's been used for years. The reason for the recent success of the paradigm in the Java ecosystem is due to the Reactive Streams specification and its implementations, like Project Reactor, RxJava, and Vert.x, which provided developers with convenient and high-level interfaces for building asynchronous and non-blocking applications without dealing with the underlying details of designing a message-driven flow. The following section will introduce Project Reactor, the reactive framework used by Spring.

8.1.2 Project Reactor: Reactive streams with Mono and Flux

Reactive Spring is based on Project Reactor, a framework for building asynchronous, non-blocking applications on the JVM. Reactor is an implementation of the *Reactive Streams* specification, and it aims to provide "a standard for asynchronous stream processing with non-blocking back pressure" (www.reactive-streams.org).

Conceptually, reactive streams resemble the Java `Stream` API in the way we use them to build data pipelines. One of the key differences is that a Java stream is pull-based: consumers process data in an imperative and synchronous fashion. Instead, reactive streams are push-based: consumers are notified by the producers when new data is available, so the processing happens asynchronously.

Reactive streams work according to a producer/consumer paradigm. Producers are called *publishers*. They produce data that might be eventually available. Reactor provides two central APIs implementing the